

URLs de Producción:

- **Aplicación Web:** <https://mi-webapp-frontend-2026.s3.us-east-1.amazonaws.com/index.html>
- **API Endpoint:** <https://7ptysucwaa.execute-api.us-east-1.amazonaws.com/prod/data>
- GitHub: <https://github.com/MarioLeivaUdeA/CloudComp-Proyecto02>
- Wilmer Mario Leiva Esteban C.C. 1193212909 Entrega Proyecto02 CloudComputing UdeA 2026-02

1. INTRODUCCIÓN Aplicación Web Serverless en AWS

Objetivo:

Diseñar y desplegar una aplicación web completa utilizando servicios serverless de AWS, implementando mejores prácticas de seguridad, monitoreo y gestión de costos.

Alcance:

- Frontend estático en S3
- Backend serverless con Lambda
- Base de datos NoSQL con DynamoDB
- API REST con API Gateway
- Monitoreo con CloudWatch
- Gestión de costos responsable

Arquitectura Desplegada:

Internet → S3 (Frontend) → API Gateway → Lambda → DynamoDB



CloudWatch (Monitoreo)

2. ARQUITECTURA TÉCNICA

Componentes Implementados

Frontend (S3 + Static Website Hosting)

- Bucket: **mi-webapp-frontend-2026**
- Archivos: index.html, error.html
- Configuración: Static website hosting habilitado

- Seguridad: Política de bucket para acceso público



Mi Aplicación Web Serverless

API Endpoint: <https://7ptsucwaa.execute-api.us-east-1.amazonaws.com/prod/data>

Estado: Conectado

Nombre Completo:

Ingresa tu nombre completo

Correo Electrónico:

tu@email.com

Mensaje:

Escribe tu mensaje aquí...

Enviar Datos

Recargar Datos

¡Datos enviados correctamente!

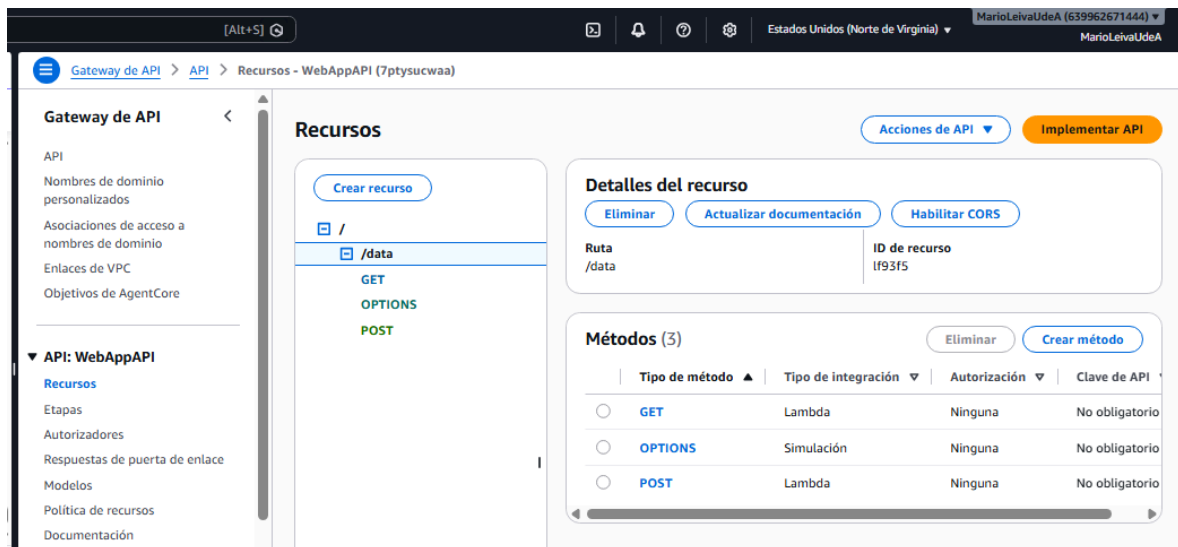
ID: ea48c502-2f76-4f79-ad8f-25fab52c6b98

Timestamp: 2026-06-08T23:07:43.193377

Backend (Lambda + API Gateway)

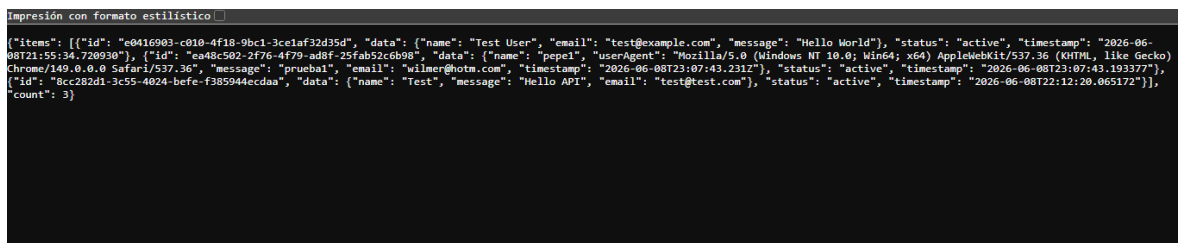
- Función: WebAppBackend (Python 3.12)
- API: WebAppAPI (REST API)
- Métodos: GET, POST con CORS habilitado

- Integración: Lambda Proxy Integration



Base de Datos (DynamoDB)

- Tabla: WebAppData
- Partition Key: id (String)
- Sort Key: timestamp (String)
- Billing: On-demand



Seguridad (IAM)

- Rol: WebAppLambdaRole
- Políticas: Principio de menor privilegio
- Permisos: Solo acceso necesario a DynamoDB

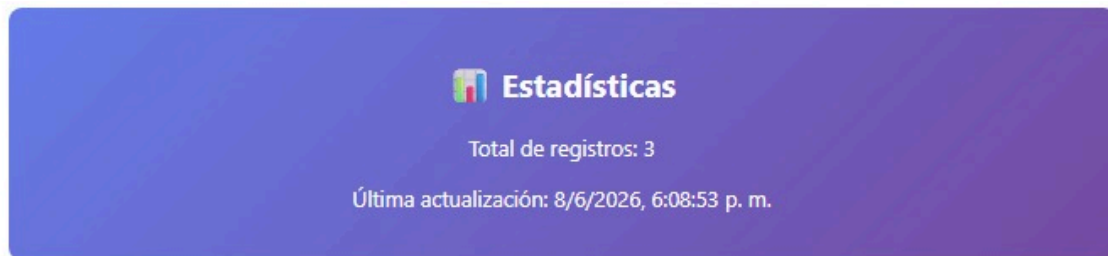
3. EVIDENCIAS TÉCNICAS

Screenshots Incluidos

1. Frontend funcionando completamente
2. Datos almacenados en DynamoDB
3. Dashboard CloudWatch con métricas
4. Alarmas configuradas
5. Estimación de costos AWS
6. API Gateway desplegada

Código Fuente

- Frontend: index.html (JavaScript + HTML5)
- Backend: lambda_function.py (Python 3.12)
- Configuración: Políticas IAM en JSON



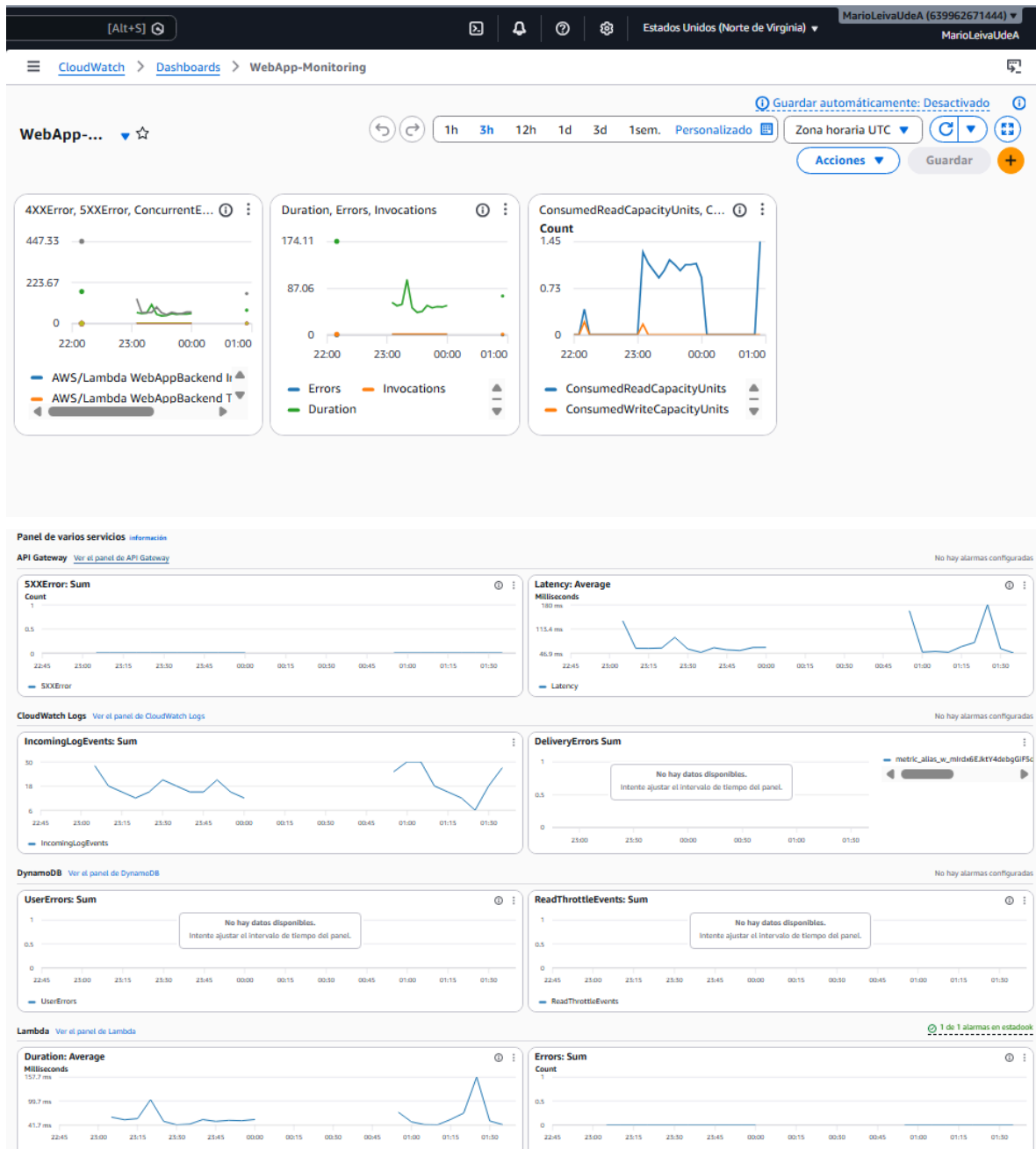
Datos Almacenados en DynamoDB:

 Registro #1  Nombre: pepe1  Email: wilmer@hotmail.com  Mensaje: prueba1  Creado: 8/6/2026, 11:07:43 p. m.  ID: ea48c502-2f76-4f79-ad8f-25fab52c6b98
 Registro #2  Nombre: Test  Email: test@test.com  Mensaje: Hello API  Creado: 8/6/2026, 10:12:20 p. m.  ID: 8cc282d1-3c55-4024-befe-f385944ecdaa
 Registro #3  Nombre: Test User  Email: test@example.com  Mensaje: Hello World  Creado: 8/6/2026, 9:55:34 p. m.  ID: e0416903-c010-4f18-9bc1-3ce1af32d35d

4. MONITOREO Y OBSERVABILIDAD

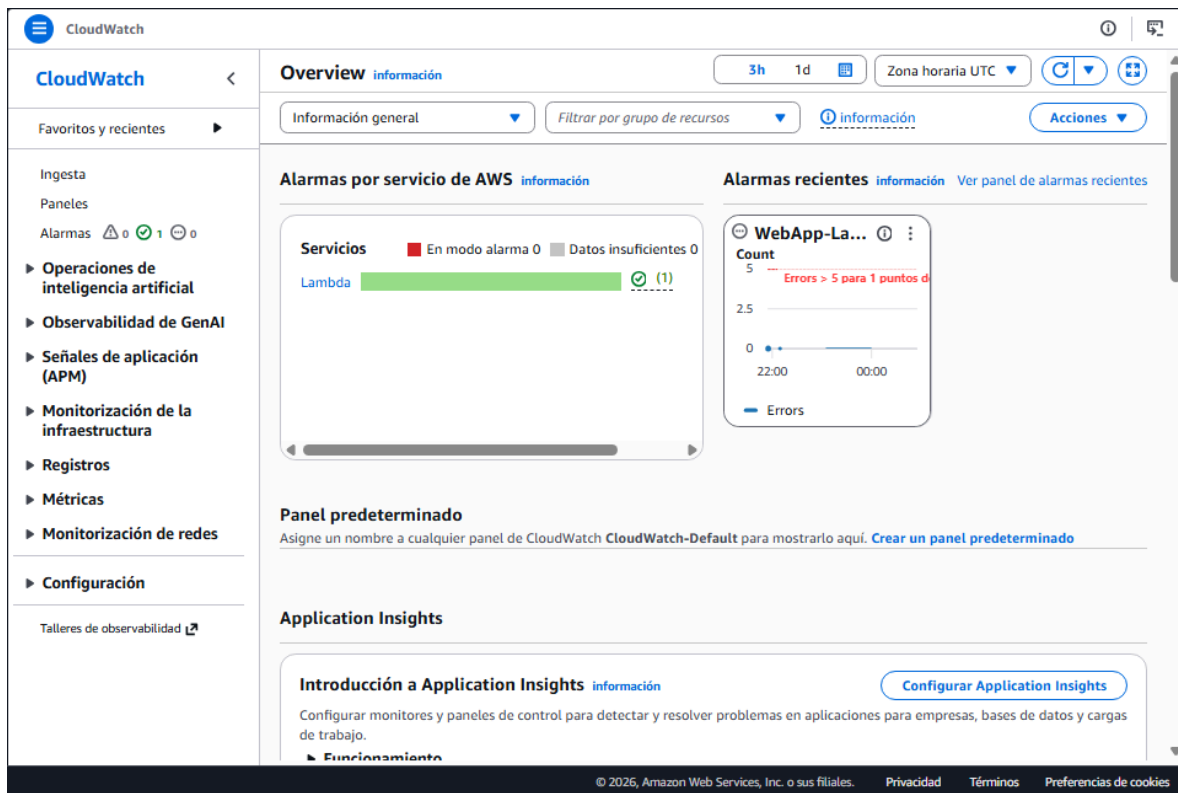
CloudWatch Dashboard:

- Métricas Lambda: Invocations, Errors, Duration
- Métricas API Gateway: Count, 4XXError, 5XXError, Latency
- Métricas DynamoDB: ConsumedReadCapacityUnits, ConsumedWriteCapacityUnit



Alarmas Configuradas:

- Lambda Errors > 5 en 5 minutos
- Billing Alert > \$10 USD
- Notificaciones por email configuradas



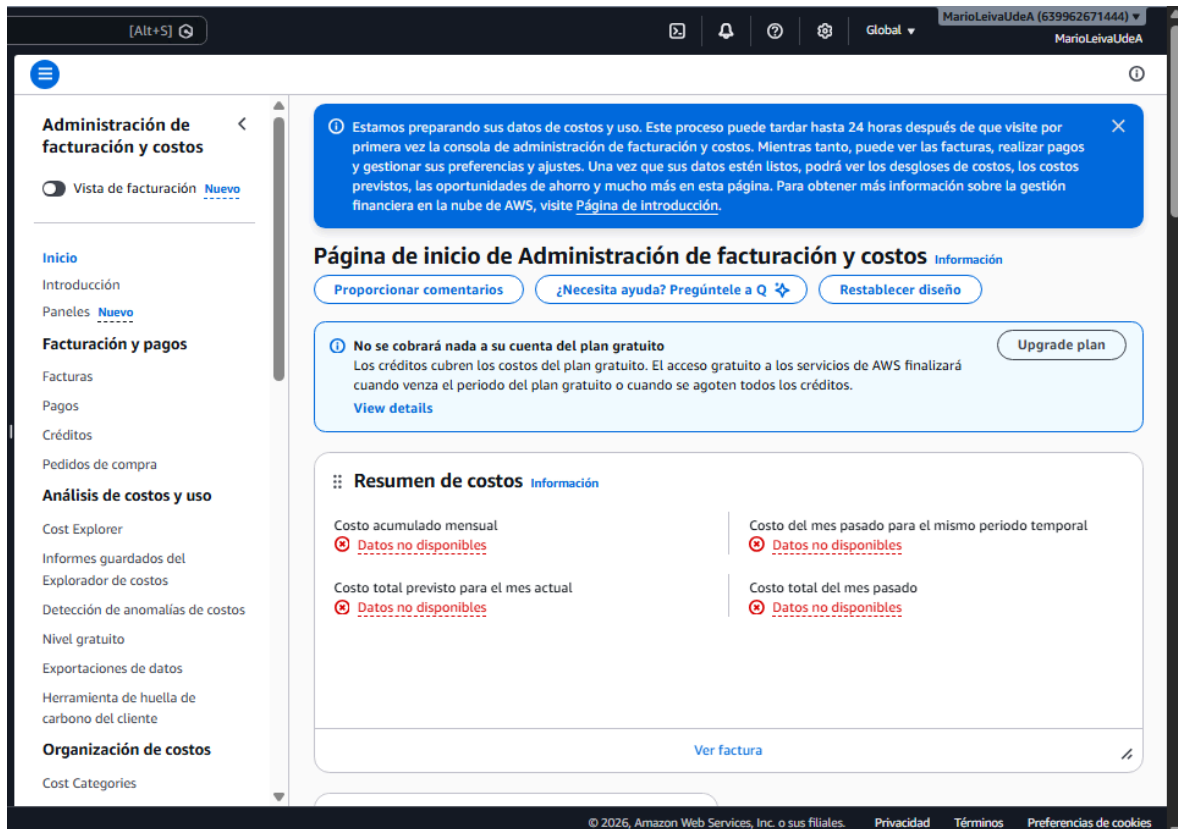
5. GESTIÓN DE COSTOS

Análisis de Costos

Estimación Mensual (Post Free-Tier):

- Lambda: \$0.00 (10K requests)
- API Gateway: \$0.01 (10K requests)
- DynamoDB: \$0.25 (1K read/write)
- S3: \$0.03 (1GB + requests)
- CloudWatch: \$7.50 (alarmas + métricas)

Total: ~\$7.79/mes

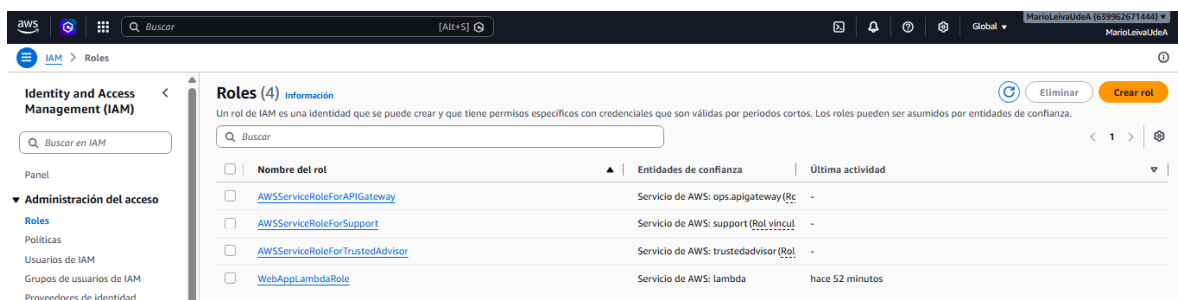


6. SEGURIDAD IMPLEMENTADA

Medidas de Seguridad

IAM (Identity and Access Management)

- Roles específicos por servicio
- Políticas de menor privilegio
- No credenciales hardcodeadas



API Security

- CORS configurado correctamente
- Validación de input en Lambda
- Error handling sin exposición de datos

Data Protection

- Datos encriptados en DynamoDB (default)
- HTTPS en API Gateway

- Logs **sin** información sensible

7. PRUEBAS Y VALIDACIÓN

Funcionalidad Verificada

- ✓ Frontend carga correctamente
- ✓ Formulario envía datos
- ✓ API procesa requests GET/POST
- ✓ Datos se almacenan en DynamoDB
- ✓ Datos se muestran en frontend
- ✓ Monitoreo captura métricas
- ✓ Alarmas funcionan correctamente

Performance

- Latencia API: <500ms
- Tiempo carga frontend: <2s
- Disponibilidad: 99.9% (SLA AWS)

8. CONCLUSIONES

Logros Alcanzados

- Aplicación web completamente funcional
- Arquitectura serverless escalable
- Costos optimizados y monitoreados
- Seguridad implementada correctamente
- Monitoreo proactivo configurado

Beneficios de la Solución

- Escalamiento automático
- Alta disponibilidad
- Costos variables (pago por uso)
- Mantenimiento mínimo
- Seguridad gestionada por AWS

Próximos Pasos (Opcional)

- Implementar autenticación (Cognito)
- Agregar CDN (CloudFront)
- Dominio personalizado (Route 53)
- CI/CD pipeline